Elementary Number Theory

Divisibility, congruences, and modular arithmetic for cryptography

Contents

1. Divisibility 4

2. The Division Algorithm 4
Division algorithm . . . 4
2.1 Why a quotient and remainder exist . . . 5
2.2 Why the quotient and remainder are unique . . . 5

3. Greatest Common Divisors 5
3.1 Greatest common divisors and remainders . . . 6

4. The Euclidean Algorithm 6
Example . . . 6

5. Bézout's Identity 7
Bézout's identity . . . 7
5.1 Proof by a least positive linear combination . . . 7
5.2 A criterion for coprimality . . . 7

6. The Extended Euclidean Algorithm 7
6.1 Keeping track of the coefficients . . . 8

7. Prime Numbers and Factorization 8
Euclid's lemma . . . 8
7.1 The fundamental theorem of arithmetic . . . 9

8. Congruences 9
8.1 Congruence is an equivalence relation . . . 10
8.2 Arithmetic with congruences . . . 10

9. Residue Classes and $\mathbb{Z}_n$ 11

10. Multiplicative Inverses Modulo n 11
Invertibility criterion . . . 11
Example . . . 12
10.1 Cancellation modulo n . . . 12

11. Linear Congruences 12
Solvability of a linear congruence . . . 12
Example . . . 13

12. Euler's Totient Function 13

13. Euler and Fermat 14
Euler's theorem . . . 14
Fermat's little theorem . . . 14

14. Fast Modular Exponentiation 14
14.1 Why the iterative algorithm works . . . 15
14.2 Example . . . 15

15. The Chinese Remainder Theorem **15**
Chinese remainder theorem . . . 16
Example . . . 16
15.1 Why the totient function is multiplicative . . . 16

16. A First View of RSA Arithmetic **17**
Toy example . . . 17

17. A Step-by-Step C++ Demonstration **18**

18. Exercises **18**
Exercise 1 — Quotients and Remainders . . . 18
Exercise 2 — Divisibility and Linear Combinations . . . 19
Exercise 3 — The Euclidean Algorithm . . . 19
Exercise 4 — Bézout Coefficients . . . 19
Exercise 5 — Euclid's Lemma . . . 19
Exercise 6 — Congruence and Remainders . . . 19
Exercise 7 — Modular Arithmetic . . . 19
Exercise 8 — Modular Inverses . . . 19
Exercise 9 — Linear Congruences . . . 19
Exercise 10 — Units and the Totient Function . . . 20
Exercise 11 — Cancellation . . . 20
Exercise 12 — Euler's Theorem . . . 20
Exercise 13 — Fast Modular Exponentiation . . . 20
Exercise 14 — The Chinese Remainder Theorem . . . 20
Exercise 15 — Toy RSA . . . 20
Exercise 16 — Degenerate Linear Congruences . . . 20
Exercise 17 — RSA for a Nonunit Message . . . 21

19. Summary **21**

The preceding chapter introduced sets, equivalence relations, functions, and methods of proof. We now apply that language to the integers. Divisibility will give us a way to compare their arithmetic structure, while congruence will provide a concrete example of an equivalence relation whose classes support arithmetic.

A cryptographic algorithm may retain only the remainder of a large power modulo some integer, and its correctness may depend on whether a modular inverse exists. Questions of this kind belong to **number theory**, the study of the arithmetic properties of integers.

This chapter develops the portion of elementary number theory that will be used repeatedly in cryptography. We begin with divisibility and greatest common divisors, derive the Euclidean and extended Euclidean algorithms, and then study prime numbers and congruences. These ideas lead to modular inverses, linear congruences, Euler's theorem, fast modular exponentiation, and the Chinese remainder theorem.

The emphasis is not merely on computation. We will state the hypotheses of each result explicitly and explain why the algorithms work, since small omissions in modular arithmetic can invalidate an otherwise plausible argument.

1. Divisibility

Let $a, b \in \mathbb{Z}$. We say that a **divides** b, written $a \mid b$, if there exists some $k \in \mathbb{Z}$ such that $b = ak$. If no such integer exists, we write $a \nmid b$.

For example, $7 \mid 42$ because $42 = 7 \cdot 6$, while $7 \nmid 40$. Every nonzero integer divides zero, since $0 = a \cdot 0$, and 1 divides every integer. The integer 0 divides only 0: if $0 \mid b$, then $b = 0k = 0$ for some $k \in \mathbb{Z}$.

The definition immediately yields several useful properties. For all $a, b, c \in \mathbb{Z}$:

- if $a \mid b$ and $b \mid c$, then $a \mid c$;
- if $a \mid b$ and $a \mid c$, then $a \mid (bx + cy)$ for every $x, y \in \mathbb{Z}$;
- if $a \mid b$, then $ac \mid bc$;
- if $a \mid b$ and $b \neq 0$, then $|a| \leq |b|$.

The second property is especially important. If $b = ar$ and $c = as$, then

$$bx + cy = a(rx + sy),$$

and $rx + sy \in \mathbb{Z}$. Thus every common divisor of b and c also divides every integer linear combination of them.

2. The Division Algorithm

Ordinary integer division produces a quotient and a remainder. The following theorem states precisely what this means.

Division algorithm

Let $a \in \mathbb{Z}$ and let $b \in \mathbb{Z}$ with $b > 0$. There exist unique integers q and r such that

$$a = bq + r, \qquad 0 \leq r < b.$$

The integer q is the **quotient**, and r is the **remainder** when a is divided by b.

For example,

$$37 = 5 \cdot 7 + 2,$$

so the quotient is 7 and the remainder is 2. Negative dividends obey the same remainder condition. Since

$$-17 = 5(-4) + 3,$$

the remainder when -17 is divided by 5 is 3, not -2.

The restriction $0 \le r < b$ is essential for uniqueness. Without it, one could write $37 = 5 \cdot 7 + 2 = 5 \cdot 6 + 7$ and obtain many different pairs.

2.1 Why a quotient and remainder exist

We use the **well-ordering principle**: every nonempty subset of $\mathbb{N}$ has a least element. This is an equivalent formulation of the induction principle used in the preceding chapter.

Fix $a \in \mathbb{Z}$ and $b > 0$, and consider the set

$$S = \{a - bk \mid k \in \mathbb{Z},\ a - bk \ge 0\}.$$

This set is nonempty. For example, choosing $k = -|a|$ gives $a + b|a| \ge 0$. Let r be the least element of S, and choose $q \in \mathbb{Z}$ such that $r = a - bq$. If $r \ge b$, then

$$r - b = a - b(q + 1) \ge 0,$$

so $r - b$ would be an element of S smaller than r, a contradiction. Therefore $0 \le r < b$, and rearranging gives the required equality $a = bq + r$.

The proof also explains the role of the remainder condition: we subtract multiples of b until another subtraction would make the remainder negative.

2.2 Why the quotient and remainder are unique

Suppose that

$$a = bq + r = bq' + r', \qquad 0 \le r, r' < b.$$

Subtracting the two representations gives

$$b(q - q') = r' - r.$$

The right-hand side satisfies $-(b - 1) \le r' - r \le b - 1$. The only multiple of b in this interval is 0, so $r' - r = 0$. Hence $r = r'$, and then $b(q - q') = 0$ implies $q = q'$. Therefore the quotient and remainder are unique.

3. Greatest Common Divisors

Let $a, b \in \mathbb{Z}$, not both zero. The **greatest common divisor** of a and b, denoted by $\gcd(a, b)$, is the unique positive integer d satisfying:

1. $d \mid a$ and $d \mid b$;
2. if $c \mid a$ and $c \mid b$, then $c \mid d$.

The second condition is stronger and more useful than merely saying that d is numerically the largest positive common divisor. It characterizes d through divisibility and continues to work cleanly in later algebraic settings.

For example, the positive common divisors of 84 and 30 are $1, 2, 3$, and 6, so

$$\gcd(84, 30) = 6.$$

Signs do not affect the greatest common divisor:

$$\gcd(a, b) = \gcd(|a|, |b|).$$

We also have $\gcd(a, 0) = |a|$ for every nonzero integer a.

Two integers a and b are **coprime**, or **relatively prime**, if $\gcd(a, b) = 1$. They need not themselves be prime; for instance, 8 and 15 are coprime.

3.1 Greatest common divisors and remainders

Suppose that $a = bq + r$. Then

$$\gcd(a, b) = \gcd(b, r).$$

Indeed, if an integer c divides both a and b, then it divides $r = a - bq$. Conversely, if c divides both b and r, then it divides $a = bq + r$. Thus (a, b) and (b, r) have exactly the same common divisors, so they have the same greatest common divisor.

This simple invariance is the basis of the Euclidean algorithm.

4. The Euclidean Algorithm

The **Euclidean algorithm** computes $\gcd(a, b)$ by repeatedly replacing a pair of integers with the divisor and remainder from the division algorithm. Let $a \geq b > 0$. Write

$$\begin{aligned} a &= bq_1 + r_1, \quad && 0 \leq r_1 < b, \\ b &= r_1 q_2 + r_2, \quad && 0 \leq r_2 < r_1, \\ r_1 &= r_2 q_3 + r_3, \quad && 0 \leq r_3 < r_2, \end{aligned}$$

and continue until a zero remainder appears. The positive remainders strictly decrease, so the process must terminate. Repeated use of the remainder property gives

$$\gcd(a, b) = \gcd(b, r_1) = \gcd(r_1, r_2) = \cdots.$$

The last nonzero remainder is therefore $\gcd(a, b)$.

Example

To compute $\gcd(252, 198)$, perform the divisions

$$\begin{aligned} 252 &= 198 \cdot 1 + 54, \\ 198 &= 54 \cdot 3 + 36, \\ 54 &= 36 \cdot 1 + 18, \\ 36 &= 18 \cdot 2 + 0. \end{aligned}$$

The last nonzero remainder is 18, so

$$\gcd(252, 198) = 18.$$

Trial division would require searching for common divisors. The Euclidean algorithm instead reduces the size of the inputs rapidly, which makes it practical even for integers far larger than those that can be factored efficiently.

5. Bézout's Identity

An **integer linear combination** of a and b is an integer of the form $ax + by$, where $x, y \in \mathbb{Z}$. The greatest common divisor can always be expressed in this form.

Bézout's identity

Let $a, b \in \mathbb{Z}$, not both zero, and let $d = \gcd(a, b)$. Then there exist integers x and y such that

$$ax + by = d.$$

The integers x and y are called **Bézout coefficients**. They are generally not unique.

5.1 Proof by a least positive linear combination

Consider the set

$$S = \{ax + by \mid x, y \in \mathbb{Z},\ ax + by > 0\}.$$

Since a and b are not both zero, either $|a|$ or $|b|$ is a positive element of S. The well-ordering principle therefore gives a least element $s = ax_0 + by_0$.

Apply the division algorithm to a with divisor s. Write $a = sq + r$, where $0 \leq r < s$. Substituting the expression for s gives

$$r = a - q(ax_0 + by_0) = a(1 - qx_0) + b(-qy_0).$$

If $r > 0$, this would be an element of S smaller than s. Thus $r = 0$, so $s \mid a$. Applying the same argument to b gives $s \mid b$.

Conversely, every common divisor c of a and b divides $ax_0 + by_0 = s$. Hence s satisfies both defining properties of $\gcd(a, b)$, proving that $s = d$ and establishing Bézout's identity. This also proves that the greatest common divisor exists for every pair of integers that are not both zero.

5.2 A criterion for coprimality

A fundamental consequence is the coprimality criterion

$$\gcd(a, b) = 1 \iff \exists x, y \in \mathbb{Z} \text{ such that } ax + by = 1.$$

The forward implication is Bézout's identity. For the reverse implication, any common divisor of a and b must divide $ax + by = 1$, so the only positive common divisor is 1.

6. The Extended Euclidean Algorithm

The Euclidean algorithm can be extended to compute Bézout coefficients as well as the greatest common divisor. After completing the ordinary divisions, substitute backward from the last nonzero remainder.

For 252 and 198, the Euclidean algorithm gave

$$18 = 54 - 36.$$

Using $36 = 198 - 3 \cdot 54$ gives

$$18 = 54 - (198 - 3 \cdot 54) = 4 \cdot 54 - 198.$$

Finally, $54 = 252 - 198$, so

$$18 = 4(252 - 198) - 198 = 4 \cdot 252 - 5 \cdot 198.$$

Thus $x = 4$ and $y = -5$ are Bézout coefficients:

$$252(4) + 198(-5) = 18.$$

This backward-substitution procedure is the **extended Euclidean algorithm**. Later we will use it to compute multiplicative inverses modulo an integer, an operation that appears directly in cryptographic key generation.

6.1 Keeping track of the coefficients

Backward substitution is convenient by hand, but an implementation can maintain the coefficients during the divisions. For positive inputs a and b, start with

$$r_0 = a = 1a + 0b,$$
$$r_1 = b = 0a + 1b.$$

Suppose two consecutive remainders have representations $r_{i-1} = s_{i-1}a + t_{i-1}b$ and $r_i = s_i a + t_i b$. If the next division uses quotient q_i, then

$$r_{i+1} = r_{i-1} - q_i r_i,$$
$$s_{i+1} = s_{i-1} - q_i s_i,$$
$$t_{i+1} = t_{i-1} - q_i t_i.$$

Substitution shows that $r_{i+1} = s_{i+1}a + t_{i+1}b$. Thus every row remains an integer linear combination of the original inputs. At the last nonzero remainder, the two stored coefficients are Bézout coefficients.

For the preceding example, the rows are

Remainder r_i	Coefficient s_i of 252	Coefficient t_i of 198
252	1	0
198	0	1
54	1	-1
36	-3	4
18	4	-5

The final row gives the same identity $18 = 4 \cdot 252 - 5 \cdot 198$ without reconstructing the calculation afterward.

7. Prime Numbers and Factorization

An integer $p > 1$ is **prime** if its only positive divisors are 1 and p. An integer $n > 1$ that is not prime is **composite**.

The number 1 is neither prime nor composite. Excluding it from the primes is necessary for unique factorization: if 1 were prime, arbitrary factors of 1 could be inserted into any factorization.

Euclid's lemma

Let p be prime and let $a, b \in \mathbb{Z}$. If $p \mid ab$, then

$$p \mid a \quad \text{or} \quad p \mid b.$$

To prove the nontrivial case, suppose $p \nmid a$. Since the only positive divisors of p are 1 and p, we have $\gcd(p, a) = 1$. Bézout's identity therefore gives integers x, y such that $px + ay = 1$. Multiplying by b yields

$$pbx + aby = b.$$

Both terms on the left are divisible by p: the first visibly, and the second because $p \mid ab$. Hence $p \mid b$.

The hypothesis that p is prime cannot simply be removed. For example, $6 \mid 2 \cdot 3$, but $6 \nmid 2$ and $6 \nmid 3$.

7.1 The fundamental theorem of arithmetic

Every integer $n > 1$ can be written as a product of primes. Moreover, this factorization is unique apart from the order of the prime factors. Equivalently, there are unique distinct primes $p_1, \ldots, p_k$ and unique positive integers $e_1, \ldots, e_k$ such that

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}, \qquad p_1 < p_2 < \cdots < p_k.$$

For example,

$$360 = 2^3 \cdot 3^2 \cdot 5.$$

To prove existence, suppose there were an integer greater than 1 with no prime factorization, and choose the least such integer n. It cannot be prime, since a prime is already a product of one prime. Hence $n = ab$ with $1 < a < n$ and $1 < b < n$. By the minimality of n, both a and b have prime factorizations, whose product is a prime factorization of n. This contradiction proves existence.

For uniqueness, suppose $n = p_1 \cdots p_r = q_1 \cdots q_s$, where all factors are prime and repetitions are allowed. Repeated application of Euclid's lemma shows that p_1 divides some q_j. Since q_j is prime, $p_1 = q_j$. Cancel these equal factors and repeat. Neither side can have factors left after the other has become 1, because a nonempty product of primes is greater than 1. Thus the prime factors agree, including their multiplicities.

Prime factorization provides conceptual formulas for greatest common divisors. For positive integers a and b, write

$$a = \prod_p p^{\alpha_p} \qquad \text{and} \qquad b = \prod_p p^{\beta_p},$$

where all but finitely many exponents are zero, then

$$\gcd(a, b) = \prod_p p^{\min(\alpha_p, \beta_p)}.$$

This formula is mathematically useful, but computing a greatest common divisor does not require factoring the inputs. That distinction matters in cryptography: the Euclidean algorithm is efficient for very large integers, while factoring appropriately chosen large integers is believed to be difficult for classical computers.

8. Congruences

Let $n \in \mathbb{Z}$ with $n \geq 2$, and let $a, b \in \mathbb{Z}$. We say that a **is congruent to** b **modulo** n, written

$$a \equiv b \pmod{n},$$

if $n \mid (a-b)$. Equivalently, there exists some $k \in \mathbb{Z}$ such that $a - b = kn$.
For example,

$$38 \equiv 3 \pmod{7}$$

because $38 - 3 = 35$ is divisible by 7. Likewise, $-2 \equiv 5 \pmod{7}$.
By the division algorithm, two integers are congruent modulo n if and only if they have the same remainder upon division by n. Congruence therefore records an integer while ignoring how many whole multiples of n it contains.

8.1 Congruence is an equivalence relation

For a fixed $n \geq 2$, congruence modulo n is an equivalence relation on $\mathbb{Z}$:

- **reflexivity:** $a \equiv a \pmod{n}$ because $n \mid (a-a) = 0$;
- **symmetry:** if $n \mid (a-b)$, then $n \mid -(a-b) = b-a$;
- **transitivity:** if $n \mid (a-b)$ and $n \mid (b-c)$, then $n \mid (a-c)$.

Thus congruence partitions $\mathbb{Z}$ into equivalence classes.

8.2 Arithmetic with congruences

Let $a, b, c, d \in \mathbb{Z}$ and $n \geq 2$. If

$$a \equiv b \pmod{n} \qquad \text{and} \qquad c \equiv d \pmod{n},$$

then

$$a + c \equiv b + d \pmod{n}, \qquad ac \equiv bd \pmod{n}.$$

The addition rule follows because

$$(a + c) - (b + d) = (a - b) + (c - d),$$

a sum of multiples of n. For multiplication, write

$$ac - bd = a(c - d) + d(a - b),$$

which is also a multiple of n. Repeated multiplication consequently gives

$$a \equiv b \pmod{n} \implies a^k \equiv b^k \pmod{n}$$

for every $k \in \mathbb{N}$.
Congruences may always be added and multiplied, but cancellation requires care. From

$$ac \equiv bc \pmod{n}$$

we may conclude $a \equiv b \pmod{n}$ only under an additional condition, such as $\gcd(c, n) = 1$. For example,

$$2 \cdot 1 \equiv 2 \cdot 4 \pmod{6},$$

but $1 \not\equiv 4 \pmod 6$.

9. Residue Classes and $\mathbb{Z}_n$

For $a \in \mathbb{Z}$ and $n \geq 2$, the **residue class** of a modulo n is

$$[a]_n = \{b \in \mathbb{Z} \mid b \equiv a \pmod n\}.$$

The set of all residue classes modulo n is

$$\mathbb{Z}_n = \{[0]_n, [1]_n, \ldots, [n-1]_n\}.$$

There are exactly n distinct classes because every integer has a unique remainder in $\{0, 1, \ldots, n-1\}$. For example,

$$[2]_5 = \{\ldots, -8, -3, 2, 7, 12, \ldots\}.$$

Addition and multiplication of residue classes are defined by

$$[a]_n + [b]_n = [a+b]_n, \qquad [a]_n[b]_n = [ab]_n.$$

These operations are **well-defined**: choosing different representatives of the same classes does not change the resulting class. This follows exactly from the arithmetic rules for congruences.

In computations, it is common to write the representative a where the class $[a]_n$ is intended. We may consequently describe arithmetic as taking place "in $\mathbb{Z}_n$." The distinction remains conceptually important: an integer is not the same object as its residue class, even when notation suppresses the brackets.

For example, the integers 2 and 7 are different, but $[2]_5 = [7]_5$. The expression $[2]_5 + [4]_5 = [1]_5$ is an equality of classes, whereas $2 + 4 = 6$ is an equality of integers. Both statements describe the same addition followed by reduction, but they take place in different sets.

10. Multiplicative Inverses Modulo n

Let $n \geq 2$. An integer a has a **multiplicative inverse modulo** n if there exists some $x \in \mathbb{Z}$ such that

$$ax \equiv 1 \pmod n.$$

In that case, x is called an inverse of a modulo n. All integers congruent to x modulo n represent the same inverse in $\mathbb{Z}_n$.

Invertibility criterion

Let $a \in \mathbb{Z}$ and $n \geq 2$. The integer a has a multiplicative inverse modulo n if and only if

$$\gcd(a, n) = 1.$$

If $\gcd(a, n) = 1$, Bézout's identity gives integers x, y such that $ax + ny = 1$. Reducing this equality modulo n gives $ax \equiv 1 \pmod n$, so x is an inverse of a.

Conversely, if $ax \equiv 1 \pmod n$, then $ax - 1 = kn$ for some $k \in \mathbb{Z}$. Hence

$$ax - kn = 1,$$

and the Bézout criterion implies $\gcd(a, n) = 1$.

The inverse is unique as a residue class. If $ax \equiv 1 \pmod n$ and $ay \equiv 1 \pmod n$, then

$$x \equiv x(ay) = (xa)y \equiv y \pmod{n}.$$

Thus the notation a^{-1} identifies a single element of $\mathbb{Z}_n$, although infinitely many integers represent that element.

Example

To find the inverse of 17 modulo 43, the extended Euclidean algorithm gives

$$1 = 2 \cdot 43 - 5 \cdot 17,$$

so the correct inverse is

$$17^{-1} \equiv -5 \equiv 38 \pmod{43}.$$

After using the extended Euclidean algorithm, it is good practice to multiply the proposed inverse and reduce it modulo n. Here $17 \cdot 38 = 646 = 15 \cdot 43 + 1$, as required.

10.1 Cancellation modulo n

Suppose $ac \equiv bc \pmod{n}$ and $\gcd(c, n) = 1$. Since c has an inverse c^{-1} modulo n, multiplication by that inverse gives

$$a \equiv b \pmod{n}.$$

The coprimality condition explains precisely when ordinary-looking cancellation is valid in modular arithmetic.

11. Linear Congruences

A **linear congruence** in the unknown x has the form

$$ax \equiv b \pmod{n},$$

where $a, b \in \mathbb{Z}$ and $n \geq 2$ are fixed.

If $\gcd(a, n) = 1$, then a has an inverse modulo n, and multiplication by a^{-1} gives the unique solution modulo n:

$$x \equiv a^{-1}b \pmod{n}.$$

The general case is governed by the greatest common divisor.

Solvability of a linear congruence

Let $d = \gcd(a, n)$. The congruence

$$ax \equiv b \pmod{n}$$

has a solution if and only if $d \mid b$. When a solution exists, there are exactly d distinct solutions modulo n.

To see the necessity, a solution satisfies $ax - b = kn$ for some integer k, so $b = ax - kn$. Since d divides both a and n, it must divide b.

If $d = n$, then $n \mid a$. In this case the congruence has no solutions when $n \nmid b$, and every residue class is a solution when $n \mid b$. This gives exactly $n = d$ solutions in the solvable case, without requiring arithmetic modulo 1.

Now suppose $d < n$ and $d \mid b$. Write $a = da'$, $b = db'$, and $n = dn'$, so that $n' \geq 2$. Then $\gcd(a', n') = 1$, and the original congruence is equivalent to

$$a'x \equiv b' \pmod{n'}.$$

This reduced congruence has one solution x_0 modulo n'. Its solutions as integers are $x = x_0 + kn'$ with $k \in \mathbb{Z}$. Consequently, the original congruence has the following solutions modulo n:

$$[x_0]_n,\ [x_0 + n']_n,\ \ldots,\ [x_0 + (d-1)n']_n.$$

These classes are distinct. If $x_0 + in' \equiv x_0 + jn' \pmod{dn'}$ for $0 \leq i, j < d$, then $d \mid (i - j)$, which forces $i = j$. Every integer k has a remainder in $\{0, \ldots, d-1\}$ modulo d, so the list also contains every solution. There are therefore exactly d solution classes.

Example

Consider

$$14x \equiv 8 \pmod{20}.$$

Since $\gcd(14, 20) = 2$ and $2 \mid 8$, solutions exist. Dividing by 2 gives

$$7x \equiv 4 \pmod{10}.$$

The inverse of 7 modulo 10 is 3, so $x \equiv 12 \equiv 2 \pmod{10}$. Therefore the two solutions modulo 20 are

$$x \equiv 2 \pmod{20} \qquad \text{and} \qquad x \equiv 12 \pmod{20}.$$

12. Euler's Totient Function

For $n \geq 1$, **Euler's totient function** $\varphi(n)$ is the number of integers in $\{1, 2, \ldots, n\}$ that are coprime to n. Equivalently, for $n \geq 2$,

$$\varphi(n) = |\{a \in \{1, 2, \ldots, n-1\} \mid \gcd(a, n) = 1\}|.$$

For example, the integers between 1 and 12 that are coprime to 12 are $1, 5, 7$, and 11, so $\varphi(12) = 4$.

If p is prime, then every integer $1 \leq a < p$ is coprime to p, and hence

$$\varphi(p) = p - 1.$$

More generally, if p is prime and $k \geq 1$, the integers between 1 and p^k that fail to be coprime to p^k are precisely the p^{k-1} multiples of p. Therefore

$$\varphi(p^k) = p^k - p^{k-1} = p^k \left(1 - \frac{1}{p}\right).$$

If m and n are coprime positive integers, then the totient function is multiplicative:

$$\varphi(mn) = \varphi(m)\varphi(n).$$

This fact follows naturally from the Chinese remainder theorem. In particular, if p and q are distinct primes, then

$$\varphi(pq) = (p-1)(q-1).$$

13. Euler and Fermat

The residue classes in $\mathbb{Z}_n$ that have multiplicative inverses are called the **units modulo** n. Their set is denoted by

$$\mathbb{Z}_n^* = \{[a]_n \in \mathbb{Z}_n \mid \gcd(a, n) = 1\}.$$

By definition, $|\mathbb{Z}_n^*| = \varphi(n)$.

Euler's theorem

Let $n \geq 2$ and let $a \in \mathbb{Z}$. If $\gcd(a, n) = 1$, then

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

To understand the proof, list representatives $r_1, \ldots, r_{\varphi(n)}$ of the units modulo n. Multiplication by a permutes these classes: if $ar_i \equiv ar_j \pmod{n}$, cancellation is valid because a is invertible, so $r_i \equiv r_j \pmod{n}$. Therefore

$$(ar_1)(ar_2)\cdots(ar_{\varphi(n)}) \equiv r_1 r_2 \cdots r_{\varphi(n)} \pmod{n}.$$

The product on the right is itself invertible, so it may be cancelled. What remains is

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

The coprimality hypothesis matters. For example, $\varphi(8) = 4$, but $2^4 = 16 \not\equiv 1 \pmod{8}$.

Fermat's little theorem

Let p be prime. If $p \nmid a$, then

$$a^{p-1} \equiv 1 \pmod{p}.$$

This is the prime-modulus case of Euler's theorem because $\varphi(p) = p - 1$. An equivalent form valid for every $a \in \mathbb{Z}$ is

$$a^p \equiv a \pmod{p}.$$

If $p \nmid a$, multiply the first form by a. If $p \mid a$, both sides of the second form are congruent to zero modulo p.

These theorems reduce certain large exponents, but only when their hypotheses are satisfied. From $a^{\varphi(n)} \equiv 1 \pmod{n}$, one may reduce an exponent modulo $\varphi(n)$ when $\gcd(a, n) = 1$; doing so without checking coprimality is not generally valid.

14. Fast Modular Exponentiation

Let $a \in \mathbb{Z}$, $n \geq 2$, and $e \in \mathbb{N}$. Cryptographic computations often require $a^e \bmod n$ when e has hundreds or thousands of bits. Constructing the integer a^e first would be wasteful. Instead, we repeatedly square and reduce modulo n after every multiplication. When $e = 0$, the result is 1; the empty product convention is used here even when $a = 0$.

Write the exponent in binary:

$$e = e_0 + 2e_1 + 2^2 e_2 + \cdots + 2^t e_t, \qquad e_i \in \{0, 1\}.$$

Then

$$a^e = \prod_{i:e_i=1} a^{2^i}.$$

The values $a^{2^i} \bmod n$ are obtained successively by squaring. This method requires a number of squarings proportional to the bit length of e, rather than a number of multiplications proportional to e itself.

14.1 Why the iterative algorithm works

For $e > 0$, initialize an accumulator $r = 1$, a current base b equal to the canonical representative of a modulo n, and a remaining exponent $u = e$. At the start of each iteration, maintain the relation

$$rb^u \equiv a^e \pmod{n}.$$

This relation is a **loop invariant**: it is true initially and remains true after each update. Write $u = 2v + \varepsilon$, where $\varepsilon \in \{0, 1\}$. Replace r by $rb^\varepsilon \bmod n$, replace b by $b^2 \bmod n$, and replace u by v. The updated values satisfy

$$(rb^\varepsilon)(b^2)^v = rb^{2v+\varepsilon} = rb^u,$$

so the invariant is preserved. When $u = 0$, it becomes $r \equiv a^e \pmod{n}$, proving that the accumulator contains the required result.

Each iteration discards one binary digit of the remaining exponent. The number of iterations is therefore the bit length of e. This counts modular arithmetic operations, not individual bit operations: the cost of each multiplication also depends on the bit length of n.

14.2 Example

Compute $7^{128} \bmod 13$. Successive squaring gives

$$\begin{aligned}
7^2 &\equiv 10 \pmod{13},\\
7^4 &\equiv 10^2 \equiv 9 \pmod{13},\\
7^8 &\equiv 9^2 \equiv 3 \pmod{13},\\
7^{16} &\equiv 3^2 \equiv 9 \pmod{13},\\
7^{32} &\equiv 9^2 \equiv 3 \pmod{13},\\
7^{64} &\equiv 3^2 \equiv 9 \pmod{13},\\
7^{128} &\equiv 9^2 \equiv 3 \pmod{13}.
\end{aligned}$$

Thus $7^{128} \equiv 3 \pmod{13}$. Fermat's little theorem provides a check: since $128 \equiv 8 \pmod{12}$ and $7^8 \equiv 3 \pmod{13}$, it gives the same result.

Fast modular exponentiation makes powers computationally manageable. It does not make the inverse problem easy: given suitable parameters and the values of a and $a^e \bmod n$, recovering e may still be difficult. This asymmetry is central to discrete-logarithm-based cryptography.

15. The Chinese Remainder Theorem

A collection of congruences may specify an integer through its remainders modulo several moduli. When the moduli are coprime, the specification is consistent and unique modulo their product.

Chinese remainder theorem

Let $n_1, n_2 \geq 2$ with $\gcd(n_1, n_2) = 1$, and let $a_1, a_2 \in \mathbb{Z}$. The system

$$\begin{aligned} x &\equiv a_1 \pmod{n_1}, \\ x &\equiv a_2 \pmod{n_2} \end{aligned}$$

has a solution. Any two solutions are congruent modulo $n_1 n_2$.

For a constructive proof, Bézout's identity gives integers u, v such that

$$un_1 + vn_2 = 1.$$

Then

$$x = a_1 v n_2 + a_2 u n_1$$

has the required remainders. Modulo n_1, the term containing n_1 vanishes and $vn_2 \equiv 1 \pmod{n_1}$; the argument modulo n_2 is symmetric.

If x and y are both solutions, then $n_1 \mid (x - y)$ and $n_2 \mid (x - y)$. Since n_1 and n_2 are coprime, their product divides $x - y$, proving uniqueness modulo $n_1 n_2$.

Example

Consider

$$x \equiv 2 \pmod{3}, \qquad x \equiv 3 \pmod{5}.$$

Since $2 \cdot 3 - 1 \cdot 5 = 1$, the construction gives

$$x = 2(-1)(5) + 3(2)(3) = 8.$$

Therefore all solutions are

$$x \equiv 8 \pmod{15}.$$

The theorem extends to any finite collection of pairwise coprime moduli. It permits arithmetic modulo a product to be decomposed into smaller independent computations, a technique used both in proofs and in efficient cryptographic implementations.

15.1 Why the totient function is multiplicative

Let $m, n \geq 2$ be coprime. The Chinese remainder theorem says that the function

$$\begin{aligned} F : \mathbb{Z}_{mn} &\longrightarrow \mathbb{Z}_m \times \mathbb{Z}_n, \\ [x]_{mn} &\longmapsto ([x]_m, [x]_n) \end{aligned}$$

is a bijection. It is well-defined because divisibility by mn implies divisibility by both m and n. Existence in the theorem gives surjectivity, while uniqueness modulo mn gives injectivity.

Moreover, $\gcd(x, mn) = 1$ if and only if $\gcd(x, m) = \gcd(x, n) = 1$. Indeed, a prime divisor common to x and either factor is also common to x and their product; conversely, a prime dividing mn divides at least one of m and n, by Euclid's lemma.

Thus F restricts to a bijection between $\mathbb{Z}_{mn}^*$ and $\mathbb{Z}_m^* \times \mathbb{Z}_n^*$. Counting elements gives

$$\varphi(mn) = |\mathbb{Z}_m^* \times \mathbb{Z}_n^*| = |\mathbb{Z}_m^*|\,|\mathbb{Z}_n^*| = \varphi(m)\varphi(n).$$

If one of the moduli is 1, the same formula follows from $\varphi(1) = 1$. This completes the justification of multiplicativity stated earlier and connects modular arithmetic directly to the Cartesian products and bijections of the preceding chapter.

16. A First View of RSA Arithmetic

The RSA cryptosystem combines several ideas from this chapter. We consider only its arithmetic correctness here; key sizes, padding, security assumptions, and implementation attacks require separate treatment.

Let p and q be distinct primes, and define

$$n = pq, \qquad \varphi(n) = (p-1)(q-1).$$

Choose an integer e with $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$. The invertibility criterion ensures that e has an inverse modulo $\varphi(n)$. Let $d \in \{1, \dots, \varphi(n) - 1\}$ satisfy

$$ed \equiv 1 \pmod{\varphi(n)}.$$

For a message representative $m \in \{0, \dots, n-1\}$, textbook RSA computes

$$c \equiv m^e \pmod{n}$$

and then

$$m' \equiv c^d \pmod{n}.$$

Since $ed = 1 + k(p-1)(q-1)$ for some nonnegative integer k, we can prove $m^{ed} \equiv m \pmod{p}$. If $p \mid m$, both sides are zero modulo p. Otherwise, Fermat's little theorem gives $m^{p-1} \equiv 1 \pmod{p}$, and hence

$$m^{ed} = m\left(m^{p-1}\right)^{k(q-1)} \equiv m \pmod{p}.$$

The same argument gives $m^{ed} \equiv m \pmod{q}$. The Chinese remainder theorem then implies

$$m^{ed} \equiv m \pmod{pq}.$$

Consequently, $m' \equiv m \pmod{n}$.

This proof applies to every message representative, including those not coprime to n. Applying Euler's theorem directly modulo n would establish correctness only when $\gcd(m, n) = 1$; splitting the argument modulo p and q removes that unnecessary restriction.

Toy example

Take $p = 5$ and $q = 11$. Then $n = 55$ and $\varphi(n) = 40$. Choose $e = 3$; the extended Euclidean algorithm gives $d = 27$ because

$$3 \cdot 27 = 81 \equiv 1 \pmod{40}.$$

For $m = 7$, encryption gives

$$c \equiv 7^3 \equiv 13 \pmod{55}.$$

Fast modular exponentiation then gives

$$13^{27} \equiv 7 \pmod{55},$$

recovering the original message representative.

This example is intentionally tiny and provides no security. Textbook RSA without a suitable padding construction is also not secure for real messages, even with large primes. The purpose of the example is solely to show how divisibility, Bézout's identity, modular inverses, Fermat's little theorem, and the Chinese remainder theorem fit together in a cryptographic construction.

17. A Step-by-Step C++ Demonstration

The complete C++ source implements the toy RSA example while exposing every important arithmetic step to the user. With no command-line arguments, it uses $p = 11$, $q = 17$, $e = 7$, and $m = 8$. Four alternative values may be supplied in that order.

From the `01-mathematics` directory, compile and run the program with

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -O2 examples/02-toy-rsa.cpp -o toy-rsa
./toy-rsa
```

The output is organized into eight explicit steps:

1. test p and q for primality by trial division;
2. compute $n = pq$ and $\varphi(n) = (p-1)(q-1)$;
3. use the Euclidean algorithm to verify $\gcd(e, \varphi(n)) = 1$;
4. display the extended Euclidean algorithm and obtain $d \equiv e^{-1} \pmod{\varphi(n)}$;
5. state the public and private key pairs;
6. encrypt m using binary modular exponentiation;
7. decrypt the resulting ciphertext using the same exponentiation algorithm;
8. compare the recovered value with the original message.

During exponentiation, the program prints the remaining exponent, its current least significant bit, the accumulated result, and the current squared base. For the default encryption, part of the trace is

```
Round 0: exponent = 7, bit = 1, result = 1, base = 8.
  bit 1: result = (1 * 8) mod 187 = 8.
  square: base = (8 * 8) mod 187 = 64.
Round 1: exponent = 3, bit = 1, result = 8, base = 64.
  bit 1: result = (8 * 64) mod 187 = 138.
```

The program validates the inputs and rejects composite or equal values for p and q, an exponent e that is not invertible modulo $\varphi(n)$, and a message outside $\{0, \ldots, n-1\}$. Its prime test is trial division, and its integer sizes are deliberately limited so that the trace remains suitable for study.

This implementation is not a model for production RSA. It has no secure padding, constant-time operations, cryptographically secure randomness, arbitrary-precision arithmetic, or protection against side-channel attacks. Real RSA implementations should use a mature cryptographic library rather than reproduce this educational code.

18. Exercises

Exercise 1 — Quotients and Remainders

For each pair (a, b), find the unique integers q and r such that $a = bq + r$ and $0 \le r < b$:

$$(a, b) = (137, 12), \qquad (a, b) = (-137, 12), \qquad (a, b) = (2026, 37).$$

Exercise 2 — Divisibility and Linear Combinations

Let $a, b, c \in \mathbb{Z}$. Prove that if $a \mid b$ and $a \mid c$, then

$$a \mid (bx + cy)$$

for every $x, y \in \mathbb{Z}$. Use the result to show that no integer greater than 1 divides both 35 and 64.

Exercise 3 — The Euclidean Algorithm

Use the Euclidean algorithm to compute

$$\gcd(12345, 6789).$$

List every division step.

Exercise 4 — Bézout Coefficients

Use the extended Euclidean algorithm to find integers x and y satisfying

$$12345x + 6789y = \gcd(12345, 6789).$$

Verify your result by direct substitution.

Exercise 5 — Euclid's Lemma

Let p be prime and let $a_1, \ldots, a_k \in \mathbb{Z}$. Prove that

$$p \mid a_1 a_2 \cdots a_k$$

implies $p \mid a_i$ for at least one $i \in \{1, \ldots, k\}$. Clearly identify where induction is used.

Exercise 6 — Congruence and Remainders

Prove that $a \equiv b \pmod{n}$ if and only if a and b have the same remainder when divided by n, where $a, b \in \mathbb{Z}$ and $n \geq 2$.

Exercise 7 — Modular Arithmetic

Compute the canonical representative in $\{0, 1, \ldots, 16\}$ of each expression modulo 17:

$$38 + 49, \qquad 38 \cdot 49, \qquad 38^4.$$

Exercise 8 — Modular Inverses

Determine whether each inverse exists. When it does, compute it using the extended Euclidean algorithm:

$$17^{-1} \pmod{3120}, \qquad 18^{-1} \pmod{3120}, \qquad 35^{-1} \pmod{64}.$$

Exercise 9 — Linear Congruences

Solve each congruence, listing all incongruent solutions modulo the stated modulus:

$$9x \equiv 12 \pmod{15},$$

and

$$14x \equiv 5 \pmod{21}.$$

If no solution exists, prove that it does not exist.

Exercise 10 — Units and the Totient Function

List every element of $\mathbb{Z}_{20}^*$ and determine $\varphi(20)$. Then verify the result from the prime-power formula and multiplicativity, and independently by counting the integers excluded because they are divisible by 2 or by 5.

Exercise 11 — Cancellation

Let $a, b, c \in \mathbb{Z}$ and $n \geq 2$. Prove directly from divisibility and Bézout's identity that

$$ac \equiv bc \pmod{n} \quad \text{and} \quad \gcd(c, n) = 1$$

imply $a \equiv b \pmod{n}$. Give a counterexample when $\gcd(c, n) \neq 1$.

Exercise 12 — Euler's Theorem

Use Euler's theorem to compute

$$7^{222} \bmod 20.$$

Explain why the theorem applies before reducing the exponent.

Exercise 13 — Fast Modular Exponentiation

Compute

$$5^{117} \bmod 19$$

using repeated squaring. Show the binary decomposition of the exponent and each modular square.

Exercise 14 — The Chinese Remainder Theorem

Find the unique solution modulo 105 to

$$\begin{aligned} x &\equiv 2 \pmod{3}, \\ x &\equiv 3 \pmod{5}, \\ x &\equiv 2 \pmod{7}. \end{aligned}$$

Verify the result in all three congruences.

Exercise 15 — Toy RSA

Let $p = 11$, $q = 17$, and $e = 7$.

1. Compute n and $\varphi(n)$.
2. Verify that e is invertible modulo $\varphi(n)$ and compute the corresponding exponent d.
3. Encrypt the message representative $m = 8$ by computing $c \equiv m^e \pmod{n}$.
4. Decrypt c by computing $c^d \bmod n$ and verify that the result is 8.

Use repeated squaring for the modular powers. This is an arithmetic exercise, not a secure choice of RSA parameters.

Exercise 16 — Degenerate Linear Congruences

Determine all solution classes modulo 7 for each congruence:

$$0x \equiv 0 \pmod 7, \qquad 0x \equiv 3 \pmod 7, \qquad 14x \equiv 21 \pmod 7.$$

Explain how each answer agrees with the criterion involving $\gcd(a, n)$. Why is it unnecessary to introduce an inverse modulo 1?

Exercise 17 — RSA for a Nonunit Message

Use $p = 5$, $q = 11$, $n = 55$, $e = 3$, and $d = 27$, but now take $m = 5$.

1. Explain why Euler's theorem cannot be applied directly to m modulo 55.
2. Compute the ciphertext c.
3. Recover $c^{27} \bmod 55$ by computing its remainders modulo 5 and 11 and applying the Chinese remainder theorem.
4. Explain which part of the RSA correctness proof covers this message.

19. Summary

This chapter developed the arithmetic of integers needed for many classical cryptographic constructions. Divisibility led to greatest common divisors, and the invariance of the greatest common divisor under remainders produced the Euclidean algorithm. Bézout's identity and the extended Euclidean algorithm then provided both a structural description of coprimality and a practical method for computing modular inverses.

Prime numbers and Euclid's lemma established the foundation for unique factorization. Congruence modulo n was introduced as an equivalence relation compatible with addition and multiplication, leading to the residue classes $\mathbb{Z}_n$, their units, and a precise criterion for cancellation and invertibility.

We solved linear congruences, defined Euler's totient function, and proved Euler's theorem and Fermat's little theorem. Fast modular exponentiation showed how enormous powers can be computed efficiently, while the Chinese remainder theorem showed how arithmetic modulo a product of coprime integers decomposes into smaller computations.

Finally, the arithmetic correctness of textbook RSA illustrated how these results interact. The next natural step is to organize familiar modular operations into algebraic structures—groups, rings, and fields—which will provide the language for discrete logarithms, finite-field arithmetic, and elliptic-curve cryptography.